Base Integer Instructions: RV32I

Category	Name	Fmt	RV32I Base		
Shifts	Shift Left Logical	R	sll	rd,rs1,rs2	rd <- rs1 << rs2, Lin=0
	Shift Left Log. Imm.	I	slli	rd,rs1,inm12	rd <- rs1 << inm, Lin=0
	Shift Right Logical	R	srl	rd,rs1,rs2	rd <- rs1 >> rs2, Rin=0
	Shift Right Log. Imm.	I	srli	rd,rs1,inm12	rd <- rs1 >> inm, Rin=0
	Shift Right **Arithmetic**	R	sra	rd,rs1,rs2	rd <- rs1 >> rs2 (Rin=signo)
	Shift Right **Arith. Imm.**	I	srai	rd,rs1,inm12	rd <- rs1 >> inm (Rin=signo)
Arithmetic	ADD	R	add	rd,rs1,rs2	rd <- rs1 + rs2
	ADD Immediate	I	addi	rd,rs1,imm12	rd <- rs1 + inmext
	SUBtract	R	sub	rd,rs1,rs2	rd <- rs1 - rs2
	Load Upper Imm	U	lui	rd,imm20	rd <-- {inm20,12'b0}
	Add Upper Imm to PC	U	auipc	rd,imm20	rd <-- PC+{inm20,12'b0}
Logical	XOR	R	xor	rd,rs1,rs2	rd <- rs1 ^ rs2
	XOR Immediate	I	xori	rd,rs1,imm12	rd <- rs1 ^ inmext
	OR	R	or	rd,rs1,rs2	rd <- rs1 | rs2
	OR Immediate	I	ori	rd,rs1,imm12	rd <- rs1 | inmext
	AND	R	and	rd,rs1,rs2	rd <- rs1 & rs2
	AND Immediate	I	andi	rd,rs1,imm12	rd <- rs1 & inmext
Compare	Set <	R	slt	rd,rs1,rs2	rd <- (rs1 <rs2)?1:0
	Set < Immediate	I	slti	rd,rs1,imm12	rd <- (rs1 <inm)?1:0
	Set < **Unsigned**	R	**sltu**	rd,rs1,rs2	rd <- (rs1 <rs2)?1:0
	Set < Imm **Unsigned**	I	**sltiu**	rd,rs1,imm	rd <- (rs1 <inm)?1:0
Branches	Branch =	B	beq	rs1,rs2,imm12	if(rs1==rs1) PC=PC+inmext;
	Branch ≠	B	bne	rs1,rs2,imm12	if(rs1!=rs1) PC=PC+inmext;
	Branch <	B	blt	rs1,rs2,imm12	if(rs1<rs1) PC=PC+inmext;
	Branch ≥	B	bge	rs1,rs2,imm12	if(rs1>=rs1) PC=PC+inmext;
	Branch < **Unsigned**	B	**bltu**	rs1,rs2,imm12	if(rs1<rs1) PC=PC+inmext;
	Branch ≥ **Unsigned**	B	**bgeu**	rs1,rs2,imm12	if(rs1>=rs1) PC=PC+inmext;
Jump & Link	J&L	J	jal	rd,imm20	rd= PC+4; PC=PC+sExt{inm20,1'b0}
	Jump & Link Register	I	jalr	rd,rs1,imm12	rd= PC+4; PC=rs1+inmext
Loads	Load Byte	I	lb	rd,inm12(rs1)	rd<- sExt(MEM[rs1+inmext]$_{7:0}$)
	Load Halfword	I	lh	rd,inm12(rs1)	rd<- sExt(MEM[rs1+inmext]$_{15:0}$)
	Load Byte **Unsigned**	I	**lbu**	rd,inm12(rs1)	rd<- zExt(MEM[rs1+inmext]$_{7:0}$)
	Load Half **Unsigned**	I	**lhu**	rd,imm12(rs1)	rd<-zsExt(MEM[rs1+inmext]$_{15:0}$)
	Load Word	I	lw	rd,inm12(rs1)	rd <- MEM[rs1+inmext]
Stores	Store Byte	S	sb	rs2,imm12(rs1)	MEM[rs1+inmext]$_{7:0}$ <- rs2$_{7:0}$
	Store Halfword	S	sh	rs2,imm12(rs1)	MEM[rs1+inmext]$_{15:0}$ <- rs2$_{15:0}$
	Store Word	S	sw	rs2,imm12(rs1)	MEM[rs1+inmext] <- rs2

Categoría	Instrucción	Ejemplo	Significado	Traducción (Real)
Movimiento de Datos	Copiar registro	`mv x5, x6`	$x5 = x6$	`addi x5, x6, 0`
	Cargar inmediato	`li x5, 123`	$x5 = 123$	`addi x5, x0, 123`
	Cargar dirección	`la x5, label`	$x5 = \&\text{label}$	`auipc` + `addi`
	No operación	`nop`	Nada	`addi x0, x0, 0`
Aritmética y Lógica	Negación arit.	`neg x5, x6`	$x5 = -x6$	`sub x5, x0, x6`
	Negación lógica	`not x5, x6`	$x5 = \sim x6$	`xori x5, x6, -1`
	Set si cero	`seqz x5, x6`	$x5 = (x6 == 0)$	`sltiu x5, x6, 1`
	Set si no cero	`snez x5, x6`	$x5 = (x6 \neq 0)$	`sltu x5, x0, x6`
Saltos Condicionales	Salta si igual 0	`beqz x5, 100`	si $(x5 == 0)$ PC+100	`beq x5, x0, 100`
	Salta si no 0	`bnez x5, 100`	si $(x5 \neq 0)$ PC+100	`bne x5, x0, 100`
	Salta si mayor	`bgt x5, x6, 100`	si $(x5 > x6)$ PC+100	`blt x6, x5, 100`
	Salta si menor	`ble x5, x6, 100`	si $(x5 \leq x6)$ PC+100	`bge x6, x5, 100`
Control de Flujo	Salto incond.	`j 100`	ir a $PC + 100$	`jal x0, 100`
	Retorno	`ret`	$PC = ra$	`jalr x0, ra, 0`
	Llamada func.	`call func`	$ra = PC + 4$; ir a $func$	`auipc` + `jalr`